

Module-II (DBMS)

The terms Serializability in DBMS

A schedule is serialized if it is equivalent to a serial schedule. A concurrent schedule must ensure it is the same as if executed serially means one after another. It refers to the sequence of actions such as read, write, abort, commit is performed in a serial manner.

Example

Let's take two transactions T1 and T2,

If both transactions are performed without interfering each other then it is called as serial schedule, it can be represented as follows –

T1	T2
READ1(A)	
WRITE1(A)	
READ1(B)	
C1	
	READ2(B)
	WRITE2(B)
	READ2(B)
	C2

Non serial schedule – When a transaction is overlapped between the transaction T1 and T2.

Example

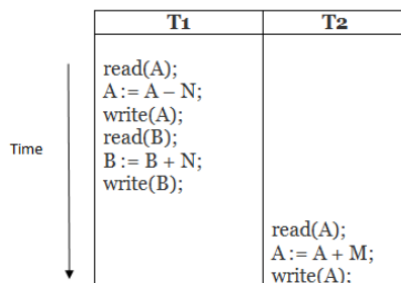
Consider the following example –

T1	T2
READ1(A)	
WRITE1(A)	
	READ2(B)
	WRITE2(B)
READ1(B)	
WRITE1(B)	
READ1(B)	

3. Serializable schedule

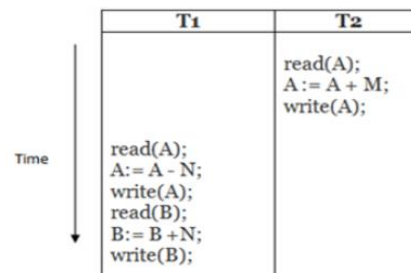
- The Serializability of schedules is used to find non-serial schedules that allow the transaction to execute concurrently without interfering with one another.
- It identifies which schedules are correct when executions of the transaction have interleaving of their operations.
- A non-serial schedule will be serializable if its result is equal to the result of its transactions executed serially.

(a)



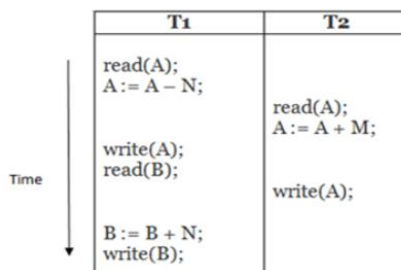
Schedule A

(b)



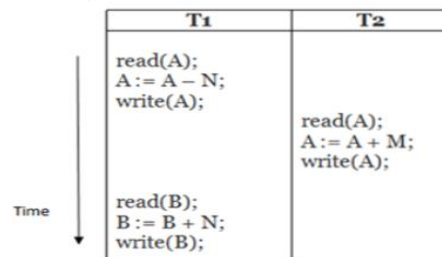
Schedule B

(c)



Schedule C

(d)



Schedule D

Schedule A and Schedule B are serial schedule.

Schedule C and Schedule D Are Non-serial schedule.

Testing of Serializability

Serialization Graph is used to test the Serializability of a schedule.

Assume a schedule S. For S, we construct a graph known as precedence graph. This graph has a pair $G = (V, E)$, where V consists a set of vertices, and E consists a set of edges. The set of vertices is used to contain all the transactions participating in the schedule. The set of edges is used to contain all edges $T_i \rightarrow T_j$ for which one of the three conditions holds:

1. Create a node $T_i \rightarrow T_j$ if T_i executes write (Q) before T_j executes read (Q).
2. Create a node $T_i \rightarrow T_j$ if T_i executes read (Q) before T_j executes write (Q).
3. Create a node $T_i \rightarrow T_j$ if T_i executes write (Q) before T_j executes write (Q).

Precedence graph for Schedule S



- If a precedence graph contains a single edge $T_i \rightarrow T_j$, then all the instructions of T_i are executed before the first instruction of T_j is executed.
- If a precedence graph for schedule S contains a cycle, then S is non-serializable. If the precedence graph has no cycle, then S is known as serializable.

For example:

	T1	T2	T3
Time ↓	Read(A)	Read(B)	
	A := f ₁ (A)		
		B := f ₂ (B) Write(B)	Read(C)
	Write(A)		C := f ₃ (C) Write(C)
		Read(A) A := f ₄ (A)	Read(B)
	Read(C) C := f ₅ (C) Write(C)	Write(A)	
		B := f ₆ (B) Write(B)	

Schedule S1

Explanation:

Read(A): In T1, no subsequent writes to A, so no new edges

Read(B): In T2, no subsequent writes to B, so no new edges

Read(C): In T3, no subsequent writes to C, so no new edges

Write(B): B is subsequently read by T3, so add edge $T2 \rightarrow T3$

Write(C): C is subsequently read by T1, so add edge $T3 \rightarrow T1$

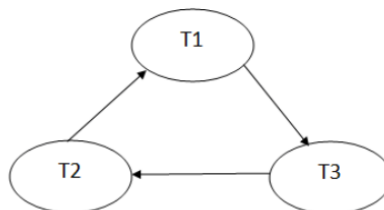
Write(A): A is subsequently read by T2, so add edge $T1 \rightarrow T2$

Write(A): In T2, no subsequent reads to A, so no new edges

Write(C): In T1, no subsequent reads to C, so no new edges

Write(B): In T3, no subsequent reads to B, so no new edges

Precedence graph for schedule S1:



Conflict Serializable Schedule

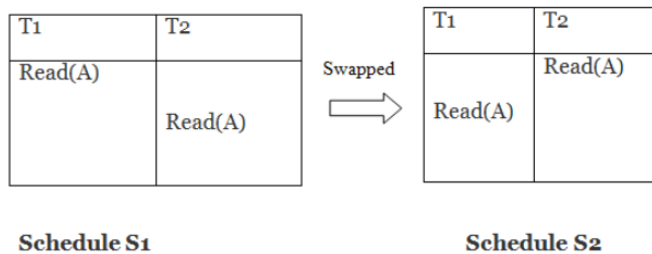
- A schedule is called conflict Serializability if after swapping of non-conflicting operations, it can transform into a serial schedule.
- The schedule will be a conflict serializable if it is conflict equivalent to a serial schedule.

Conflicting Operations

The two operations become conflicting if all conditions satisfy:

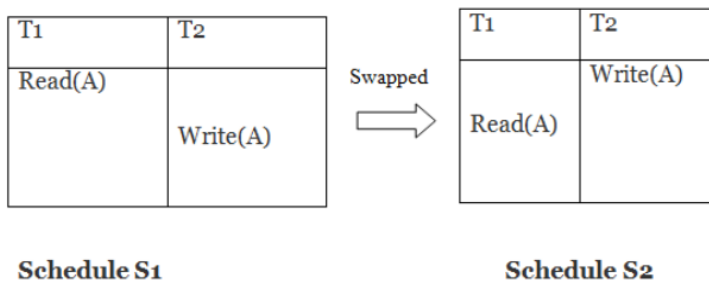
1. Both belong to separate transactions.
2. They have the same data item.
3. They contain at least one write operation.

1. T1: Read(A) T2: Read(A)



Here, $S1 = S2$. That means it is non-conflict.

2. T1: Read(A) T2: Write(A)



Here, $S1 \neq S2$. That means it is conflict.

Conflict Equivalent

In the conflict equivalent, one can be transformed to another by swapping non-conflicting operations. In the given example, S2 is conflict equivalent to S1 (S1 can be converted to S2 by swapping non-conflicting operations).

Two schedules are said to be conflict equivalent if and only if:

1. They contain the same set of the transaction.
2. If each pair of conflict operations are ordered in the same way.

Example:

Non-serial schedule		Serial Schedule	
T1	T2	T1	T2
Read(A) Write(A)		Read(A) Write(A)	
	Read(A) Write(A)	Read(B) Write(B)	
Read(B) Write(B)			Read(A) Write(A)
	Read(B) Write(B)		Read(B) Write(B)
Schedule S1		Schedule S2	

Schedule S2 is a serial schedule because, in this, all operations of T1 are performed before starting any operation of T2. Schedule S1 can be transformed into a serial schedule by swapping non-conflicting operations of S1.

After swapping of non-conflict operations, the schedule S1 becomes:

T1	T2
Read(A) Write(A) Read(B) Write(B)	
	Read(A) Write(A) Read(B) Write(B)

Since, S1 is conflict serializable.

View Serializability

A schedule is view-Serializability if it is viewed equivalent to a serial schedule.

The rules it follows are as follows –

- T1 is reading the initial value of A, then T2 also reads the initial value of A.
- T1 is the reading value written by T2, then T2 also reads the value written by T1.
- T1 is writing the final value, and then T2 also has the write operation as the final value.

View Equivalent

Two schedules S1 and S2 are said to be view equivalent if they satisfy the following conditions:

1. Initial Read

An initial read of both schedules must be the same. Suppose two schedule S1 and S2. In schedule S1, if a transaction T1 is reading the data item A, then in S2, transaction T1 should also read A.

T1	T2
Read(A)	Write(A)

Schedule S1

T1	T2
Read(A)	Write(A)

Schedule S2

Above two schedules are view equivalent because Initial read operation in S1 is done by T1 and in S2 it is also done by T1.

2. Updated Read

In schedule S1, if Ti is reading A which is updated by Tj then in S2 also, Ti should read A which is updated by Tj.

T1	T2	T3
Write(A)	Write(A)	Read(A)

Schedule S1

T1	T2	T3
Write(A)	Write(A)	<u>Read(A)</u>

Schedule S2

Above two schedules are not view equal because, in S1, T3 is reading A updated by T2 and in S2, T3 is reading A updated by T1.

3. Final Write

A final write must be the same between both the schedules. In schedule S1, if a transaction T1 updates A at last then in S2, final writes operations should also be done by T1.

T1	T2	T3
Write(A)	Read(A)	
		Write(A)

Schedule S1

T1	T2	T3
Write(A)	Read(A)	
		Write(A)

Schedule S2

Above two schedules is view equal because Final write operation in S1 is done by T3 and in S2, the final write operation is also done by T3.

Example:

T1	T2	T3
Read(A)		
Write(A)	Write(A)	
		Write(A)

Schedule S

With 3 transactions, the total number of possible schedule

$$= 3! = 6$$

S1 = <T1 T2 T3>

S2 = <T1 T3 T2>

S3 = <T2 T3 T1>

S4 = <T2 T1 T3>

S5 = <T3 T1 T2>

S6 = <T3 T2 T1>

Taking first schedule S1:

T1	T2	T3
Read(A)		
Write(A)		
	Write(A)	
		Write(A)

Schedule S1

Step 1: final updating on data items

In both schedules S and S1, there is no read except the initial read that's why we don't need to check that condition.

Step 2: Initial Read

The initial read operation in S is done by T1 and in S1, it is also done by T1.

Step 3: Final Write

The final write operation in S is done by T3 and in S1, it is also done by T3. So, S and S1 are view Equivalent.

The first schedule S1 satisfies all three conditions, so we don't need to check another schedule.

Hence, view equivalent serial schedule is:

T1 → T2 → T3

Recoverability of Schedule

Sometimes a transaction may not execute completely due to a software issue, system crash or hardware failure. In that case, the failed transaction has to be rollback. But some other transaction may also have used value produced by the failed transaction. So we also have to rollback those transactions.

T1	T1's buffer space	T2	T2's buffer space	Database
				A = 6500
Read(A);	A = 6500			A = 6500
A = A - 500;	A = 6000			A = 6500
Write(A);	A = 6000			A = 6000
		Read(A);	A = 6000	A = 6000
		A = A + 1000;	A = 7000	A = 6000
		Write(A);	A = 7000	A = 7000
		Commit;		
Failure Point				
Commit;				

The above table 1 shows a schedule which has two transactions. T1 reads and writes the value of A and that value is read and written by T2. T2 commits but later on, T1 fails. Due to the failure, we have to rollback T1. T2 should also be rollback because it reads the value written by T1, but T2 can't be rollback because it already committed. So this type of schedule is known as irrecoverable schedule.

Irrecoverable schedule: The schedule will be irrecoverable if Tj reads the updated value of Ti and Tj committed before Ti commit.

T1	T1's buffer space	T2	T2's buffer space	Database
				A = 6500
Read(A);	A = 6500			A = 6500
A = A - 500;	A = 6000			A = 6500
Write(A);	A = 6000			A = 6000
		Read(A);	A = 6000	A = 6000
		A = A + 1000;	A = 7000	A = 6000
		Write(A);	A = 7000	A = 7000
Failure Point				
Commit;				
		Commit;		

The above table 2 shows a schedule with two transactions. Transaction T1 reads and writes A, and that value is read and written by transaction T2. But later on, T1 fails. Due to this, we have to rollback T1. T2 should be rollback because T2 has read the value written by T1. As it has not committed before T1 commits so we can rollback transaction T2 as well. So it is recoverable with cascade rollback.

Recoverable with cascading rollback: The schedule will be recoverable with cascading rollback if Tj reads the updated value of Ti. Commit of Tj is delayed till commit of Ti.

T1	T1's buffer space	T2	T2's buffer space	Database
				A = 6500
Read(A);	A = 6500			A = 6500
A = A - 500;	A = 6000			A = 6500
Write(A);	A = 6000			A = 6000
Commit;		Read(A);	A = 6000	A = 6000
		A = A + 1000;	A = 7000	A = 6000
		Write(A);	A = 7000	A = 7000
		Commit;		

The above Table 3 shows a schedule with two transactions. Transaction T1 reads and write A and commits, and that value is read and written by T2. So this is a cascade less recoverable schedule.

Failure Classification

To find that where the problem has occurred, we generalize a failure into the following categories:

1. Transaction failure

2. System crash
3. Disk failure

1. Transaction failure

The transaction failure occurs when it fails to execute or when it reaches a point from where it can't go any further. If a few transaction or process is hurt, then this is called as transaction failure.

Reasons for a transaction failure could be -

1. **Logical errors:** If a transaction cannot complete due to some code error or an internal error condition, then the logical error occurs.
2. **Syntax error:** It occurs where the DBMS itself terminates an active transaction because the database system is not able to execute it.

For example, the system aborts an active transaction, in case of deadlock or resource unavailability.

2. System Crash

- System failure can occur due to power failure or other hardware or software failure. **Example:** Operating system error.

3. Disk Failure

- It occurs where hard-disk drives or storage drives used to fail frequently. It was a common problem in the early days of technology evolution.
- Disk failure occurs due to the formation of bad sectors, disk head crash, and unreachability to the disk or any other failure, which destroy all or part of disk storage.

Query Processing in DBMS

Query Processing is the activity performed in extracting data from the database. In query processing, it takes various steps for fetching the data from the database. The steps involved are:

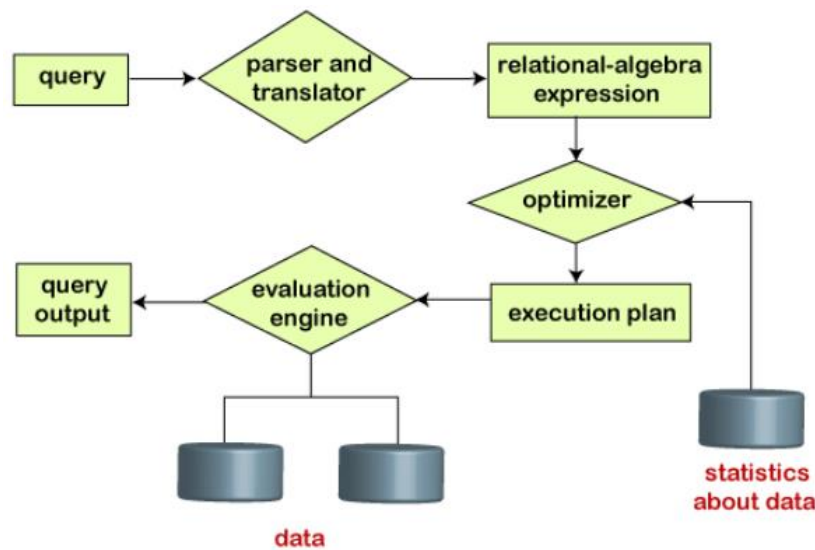
1. Parsing and translation
2. Optimization

3. Evaluation

The query processing works in the following way:

Parsing and Translation

As query processing includes certain activities for data retrieval. Initially, the given user queries get translated in high-level database languages such as SQL. When a user executes any query, for generating the internal form of the query, the parser in the system checks the syntax of the query, verifies the name of the relation in the database, the tuple, and finally the required attribute value.



Steps in query processing

In SQL, a user wants to fetch the records of the employees whose salary is greater than or equal to 10000. For doing this, the following query is undertaken:

select emp_name from Employee where salary>10000;

Thus, to make the system understand the user query, it needs to be translated in the form of relational algebra. We can bring this query in the relational algebra form as:

- $\sigma_{\text{salary} > 10000} (\pi_{\text{salary}} (\text{Employee}))$
- $\pi_{\text{salary}} (\sigma_{\text{salary} > 10000} (\text{Employee}))$

After translating the given query, we can execute each relational algebra operation by using different algorithms. So, in this way, a query processing begins its working.

Optimization

- The cost of the query evaluation can vary for different types of queries. Although the system is responsible for constructing the evaluation plan, the user does need not to write their query efficiently.
- Usually, a database system generates an efficient query evaluation plan, which minimizes its cost. This type of task performed by the database system and is known as Query Optimization.
- For optimizing a query, the query optimizer should have an estimated cost analysis of each operation. It is because the overall operation cost depends on the memory allocations to several operations, execution costs, and so on.

Evaluation

For this, with addition to the relational algebra translation, it is required to annotate the translated relational algebra expression with the instructions used for specifying and evaluating each operation. Thus, after translating the user query, the system executes a query evaluation plan.

Query Evaluation Plan

- In order to fully evaluate a query, the system needs to construct a query evaluation plan.
- The annotations in the evaluation plan may refer to the algorithms to be used for the particular index or the specific operations.
- Such relational algebra with annotations is referred to as **Evaluation Primitives**. The evaluation primitives carry the instructions needed for the evaluation of the operation.

- Thus, a query evaluation plan defines a sequence of primitive operations used for evaluating a query. The query evaluation plan is also referred to as the query execution plan.

Query Execution Engine:

Query Execution Engine is responsible for generating the output of the given query. It takes the query execution plan, executes it, and finally makes the output for the user query.

Cost-based Query Optimization

The DBMS's optimizer will use an internal cost model to estimate the execution cost for a particular query plan. This provides an estimate to determine whether one plan is better than another without having to actually run the query.

from estimating the usage of different resources:

- **CPU:** Small cost; tough to estimate.
- **Disk:** Number of block transferred.
- **Memory:** Amount of DRAM used.
- **Network:** Number of messages transfer ed

To accomplish this, the DBMS stores internal statistics about tables, attributes, and indexes in its internal catalog. Different systems update the statistics at different times. Commercial DBMS have way more robust and accurate statistics compared to the open source systems. These are estimates and thus the cost estimates will often be inaccurate.

Total Cost:

Summation of all cost factors

Total cost = CPU cost + I/O cost + communication cost

CPU cost = unit instruction cost * no. of instructions

I/O cost = unit disk I/O cost * no. of disk I/Os

communication cost = message initiation + transmission

Response Time:

Elapsed time between the initiation and the completion of a query

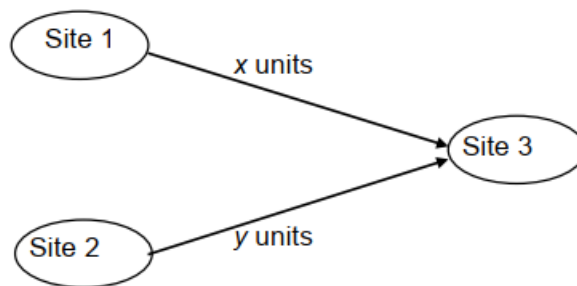
Response time = CPU time + I/O time + communication time

CPU time = unit instruction time * no. of sequential instructions

I/O time = unit I/O time * no. of sequential I/Os

communication time = unit msg initiation time * no. of sequential msg + unit transmission time * no. of sequential bytes

Example



Assume that only the communication cost is considered

Total time = 2 message initialization time + unit transmission time * (x+y)

Response time = max {time to send x from 1 to 3, time to send y from 2 to 3}

Time to send x from 1 to 3 = message initialization time + unit transmission time * x

Time to send y from 2 to 3 = message initialization time + unit transmission time * y

Measures of Query Cost in DBMS

Query Cost is a cost in which the enhancer considers what amount of time your query will require (comparative with absolute clump time). Then the analyser attempts to pick the most ideal query plan by taking a glance at your inquiry and insights of your information, attempting a few execution designs, and choosing the most inexpensive of them.

The measures of query cost in DBMS can be done by creating a framework that can make numerous designs for an inquiry. It tends to be finished by the means of contrasting every conceivable arrangement as far as their assessed cost. For working out the net assessed cost of any arrangement, the expense of every activity inside an arrangement ought to be set in a deterministic and consolidated cost to get the net assessed cost of the query assessment plan.

Example: We utilize the number of square exchanges that is basically the block from the disk and the quantity of the disk seeks to appraise the expense of a query assessment plan. Assuming that the disk subsystem takes a normal of t_T seconds to move a square of information and has a normal block access time (disk lookup time in addition to rotational idleness) of t_S seconds, then, at that point, an activity that moves **b** obstructs and performs **S** looks for would take $b * t_T + S * t_S$ seconds. The upsides of t_T and t_S should be aligned for the disk framework utilization, however, normal qualities for top-end disk today would be $t_S = 4$ milliseconds and $t_T = 0.1$ milliseconds, expecting a 4-kilobyte block size and an exchange pace of 40 megabytes each second.

t_T – time to transfer one block

t_S – time for one to seek

Cost for b block transfers plus S seeks

$$b * t_T + S * t_S$$

The expense assessment of a query assessment plan is determined the different assets that follow as:

1. The number of disk accesses.
2. Time of Execution taken by the CPU to execute a query.
3. The involved Communication costs in either distributed or parallel database systems.

Concurrency Control

Concurrency Control is the working concept that is required for controlling and managing the concurrent execution of database operations and thus avoiding the inconsistencies in the database. Thus, for maintaining the concurrency of the database, we have the concurrency control protocols.

Concurrency Control Protocols

The concurrency control protocols ensure the atomicity, consistency, isolation, durability and Serializability of the concurrent execution of the database transactions. Therefore, these protocols are categorized as:

- *Lock Based Concurrency Control Protocol*
- *Time Stamp Concurrency Control Protocol*
- *Validation Based Concurrency Control Protocol*

We will understand and discuss each protocol one by one in our next sections.

Lock-Based Protocol

In this type of protocol, any transaction cannot read or write data until it acquires an appropriate lock on it. There are two types of lock:

1. Shared lock:

- It is also known as a Read-only lock. In a shared lock, the data item can only read by the transaction.
- It can be shared between the transactions because when the transaction holds a lock, then it can't update the data on the data item.

2. Exclusive lock:

- In the exclusive lock, the data item can be both reads as well as written by the transaction.
- This lock is exclusive, and in this lock, multiple transactions do not modify the same data simultaneously

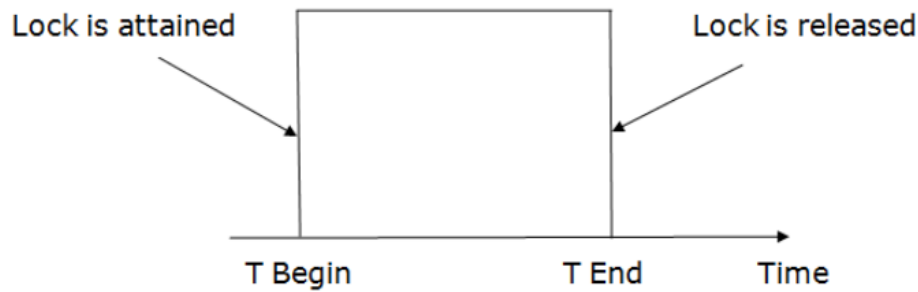
There are four types of lock protocols available:

1. Simplistic lock protocol

It is the simplest way of locking the data while transaction. Simplistic lock-based protocols allow all the transactions to get the lock on the data before insert or delete or update on it. It will unlock the data item after completing the transaction.

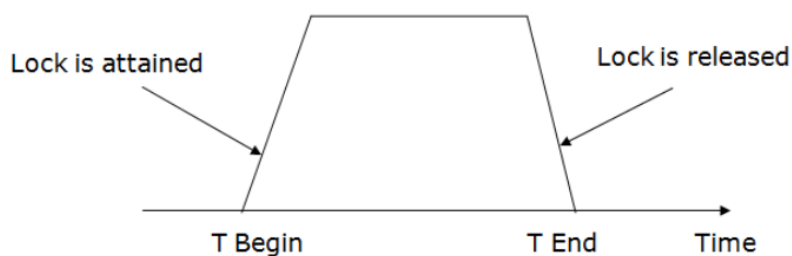
2. Pre-claiming Lock Protocol

- Pre-claiming Lock Protocols evaluate the transaction to list all the data items on which they need locks.
- Before initiating an execution of the transaction, it requests DBMS for all the lock on all those data items.
- If all the locks are granted, then this protocol allows the transaction to begin. When the transaction is completed then it releases all the lock.
-



3. Two-phase locking (2PL)

- The two-phase locking protocol divides the execution phase of the transaction into three parts.
- In the first part, when the execution of the transaction starts, it seeks permission for the lock it requires.
- In the second part, the transaction acquires all the locks. The third phase is started as soon as the transaction releases its first lock.
- In the third phase, the transaction cannot demand any new locks. It only releases the acquired locks.



There are two phases of 2PL:

Growing phase: In the growing phase, a new lock on the data item may be acquired by the transaction, but none can be released.

Shrinking phase: In the shrinking phase, existing lock held by the transaction may be released, but no new locks can be acquired.

In the below example, if lock conversion is allowed then the following phase can happen:

1. Upgrading of lock (from S(a) to X (a)) is allowed in growing phase.
2. Downgrading of lock (from X(a) to S(a)) must be done in shrinking phase.

Example:

	T1	T2
0	LOCK-S(A)	
1		LOCK-S(A)
2	LOCK-X(B)	
3	——	——
4	UNLOCK(A)	
5		LOCK-X(C)
6	UNLOCK(B)	
7		UNLOCK(A)
8		UNLOCK(C)
9	——	——

The following way shows how unlocking and locking work with 2-PL.

Transaction T1:

- **Growing phase:** from step 1-3
- **Shrinking phase:** from step 5-7
- **Lock point:** at 3

Transaction T2:

- **Growing phase:** from step 2-6
- **Shrinking phase:** from step 8-9
- **Lock point:** at 6

Timestamp Ordering Protocol

- The Timestamp Ordering Protocol is used to order the transactions based on their Timestamps. The order of transaction is nothing but the ascending order of the transaction creation.
- The priority of the older transaction is higher that's why it executes first. To determine the timestamp of the transaction, this protocol uses system time or logical counter.
- The lock-based protocol is used to manage the order between conflicting pairs among transactions at the execution time. But Timestamp based protocols start working as soon as a transaction is created.
- Let's assume there are two transactions T1 and T2. Suppose the transaction T1 has entered the system at 007 times and transaction T2 has entered the system at 009 times. T1 has the higher priority, so it executes first as it is entered the system first.

- The timestamp ordering protocol also maintains the timestamp of last 'read' and 'write' operation on a data.

Basic Timestamp ordering protocol works as follows:

1. Check the following condition whenever a transaction T_i issues a **Read (X)** operation:

- If $W_TS(X) > TS(T_i)$ then the operation is rejected.
- If $W_TS(X) \leq TS(T_i)$ then the operation is executed.
- Timestamps of all the data items are updated.

2. Check the following condition whenever a transaction T_i issues a **Write(X)** operation:

- If $TS(T_i) < R_TS(X)$ then the operation is rejected.
- If $TS(T_i) < W_TS(X)$ then the operation is rejected and T_i is rolled back otherwise the operation is executed.

Where,

TS(TI) denotes the timestamp of the transaction T_i .

R_TS(X) denotes the Read time-stamp of data-item X .

W_TS(X) denotes the Write time-stamp of data-item X .

Validation Based Protocol

Validation phase is also known as optimistic concurrency control technique. In the validation based protocol, the transaction is executed in the following three phases:

1. **Read phase:** In this phase, the transaction T is read and executed. It is used to read the value of various data items and stores them in temporary local variables. It can perform all the write operations on temporary variables without an update to the actual database.
2. **Validation phase:** In this phase, the temporary variable value will be validated against the actual data to see if it violates the Serializability.
3. **Write phase:** If the validation of the transaction is validated, then the temporary results are written to the database or system otherwise the transaction is rolled back.

Here each phase has the following different timestamps:

Start(T_i): It contains the time when T_i started its execution.

Validation (T_i): It contains the time when T_i finishes its read phase and starts its validation phase.

Finish(T_i): It contains the time when T_i finishes its write phase.

- This protocol is used to determine the time stamp for the transaction for serialization using the time stamp of the validation phase, as it is the actual phase which determines if the transaction will commit or rollback.
- Hence $TS(T) = \text{validation}(T)$.
- The Serializability is determined during the validation process. It can't be decided in advance.
- While executing the transaction, it ensures a greater degree of concurrency and also less number of conflicts.
- Thus it contains transactions which have less number of rollbacks.

Database Security

Security of databases refers to the array of controls, tools, and procedures designed to ensure and safeguard confidentiality, integrity, and accessibility. This will concentrate on confidentiality because it's a component that is most at risk in data security breaches.

Security for databases must cover and safeguard the following aspects:

- The database containing data.
- Database management systems (DBMS)
- Any applications that are associated with it.
- Physical database servers or the database server virtual, and the hardware that runs it.

- The infrastructure for computing or network that is used to connect to the database.

Why Database Security is Important?

According to the definition, a data breach refers to a breach of data integrity in databases. The amount of damage an incident like a data breach can cause our business is contingent on various consequences or elements.

- **Intellectual property that is compromised:** Our intellectual property--trade secrets, inventions, or proprietary methods -- could be vital for our ability to maintain an advantage in our industry. If our intellectual property has been stolen or disclosed and our competitive advantage is lost, it could be difficult to keep or recover.
- **The damage to our brand's reputation:** Customers or partners may not want to purchase goods or services from us (or deal with our business) If they do not feel they can trust our company to protect their data or their own.
- **The concept of business continuity (or lack of it):** Some businesses cannot continue to function until a breach has been resolved.
- **Penalties or fines to be paid for not complying:** The cost of not complying with international regulations like the Sarbanes-Oxley Act (SAO) or Payment Card Industry Data Security Standard (PCI DSS) specific to industry regulations on data privacy, like HIPAA or regional privacy laws like the European Union's General Data Protection Regulation (GDPR) could be a major problem with fines in worst cases in excess of many million dollars for each violation.
- **Costs for repairing breaches and notifying consumers about them:** Alongside notifying customers of a breach, the company that has been

breached is required to cover the investigation and forensic services such as crisis management, triage repairs to the affected systems, and much more.

Common Threats and Challenges

Numerous software configurations that are not correct, weaknesses, or patterns of carelessness or abuse can lead to a breach of security. Here are some of the most prevalent kinds of reasons for security attacks and the reasons.

Insider Dangers

An insider threat can be an attack on security from any three sources having an access privilege to the database.

- A malicious insider who wants to cause harm
- An insider who is negligent and makes mistakes that expose the database to attack. vulnerable to attacks
- An infiltrator is an outsider who acquires credentials by using a method like phishing or accessing the database of credential information in the database itself.

Insider dangers are among the most frequent sources of security breaches to databases. They often occur as a consequence of the inability of employees to have access to privileged user credentials.

Human Error

The unintentional mistakes, weak passwords or sharing passwords, and other negligent or uninformed behaviours of users remain the root causes of almost half (49 percent) of all data security breaches.

Database Software Vulnerabilities can be Exploited

Hackers earn their money by identifying and exploiting vulnerabilities in software such as databases management software. The major database software companies and open-source databases management platforms release regular security patches to fix these weaknesses. However, failing to implement the patches on time could increase the risk of being hacked.

SQL/NoSQL Injection Attacks

A specific threat to databases is the infusing of untrue SQL as well as other non-SQL string attacks in queries for databases delivered by web-based apps and HTTP headers. Companies that do not follow the safe coding practices for web applications and conduct regular vulnerability tests are susceptible to attacks using these.

DDoS (DoS/DDoS) Attacks

In a denial-of-service (DoS) attack in which the attacker overwhelms the targeted server -- in this case, the database server with such a large volume of requests that the server is unable to meet no longer legitimate requests made by actual users. In most cases, the server is unstable or even fails to function.

Malware

Malware is software designed to exploit vulnerabilities or cause harm to databases. Malware can be accessed via any device that connects to the databases network.

Nested Queries in SQL

Nested queries are a way to perform complex queries by embedding one query within another. The outer query can apply some conditions on the results of the inner query. Let us use **STUDENT**, **COURSE**, **STUDENT_COURSE** tables for understanding nested queries.

STUDENT

S_ID	S_NAME	S_ADDRESS	S_PHONE	S_AGE
S1	RAM	DELHI	9455123451	18
S2	RAMESH	GURGAON	9652431543	18
S3	SUJIT	ROHTAK	9156253131	20
S4	SURESH	DELHI	9156768971	18

COURSE

C_ID	C_NAME
C1	DSA
C2	Programming
C3	DBMS

STUDENT_COURSE

S_ID	C_ID
S1	C1
S1	C3
S2	C1
S3	C2
S4	C2
S4	C3

There are mainly two types of nested queries:

- **Independent Nested Queries:** In independent nested queries, query execution starts from innermost query to outermost queries. The execution of inner query is independent of outer

query, but the result of inner query is used in execution of outer query. Various operators like IN, NOT IN, ANY, ALL etc are used in writing independent nested queries.

IN: If we want to find out **S_ID** who are enrolled in **C_NAME** 'DSA' or 'DBMS', we can write it with the help of independent nested query and IN operator. From **COURSE** table, we can find out **C_ID** for **C_NAME** 'DSA' or 'DBMS' and we can use these **C_ID**s for finding **S_ID**s from **STUDENT_COURSE** TABLE. **STEP 1:** Finding **C_ID** for **C_NAME** = 'DSA' or 'DBMS' Select **C_ID** from **COURSE** where **C_NAME** = 'DSA' or **C_NAME** = 'DBMS' **STEP 2:** Using **C_ID** of step 1 for finding **S_ID** Select **S_ID** from **STUDENT_COURSE** where **C_ID** IN (SELECT **C_ID** from **COURSE** where **C_NAME** = 'DSA' or **C_NAME**='DBMS'); The inner query will return a set with members C1 and C3 and outer query will return those **S_ID**s for which **C_ID** is equal to any member of set (C1 and C3 in this case). So, it will return S1, S2 and S4. **Note:** If we want to find out names of **STUDENT**s who have either enrolled in 'DSA' or 'DBMS', it can be done as: Select **S_NAME** from **STUDENT** where **S_ID** IN (Select **S_ID** from **STUDENT_COURSE** where **C_ID** IN (SELECT **C_ID** from **COURSE** where **C_NAME**='DSA' or **C_NAME**='DBMS'));

NOT IN: If we want to find out **S_ID**s of **STUDENT**s who have neither enrolled in 'DSA' nor in 'DBMS', it can be done as: Select **S_ID** from **STUDENT** where **S_ID** NOT IN (Select **S_ID** from **STUDENT_COURSE** where **C_ID** IN (SELECT **C_ID** from **COURSE** where **C_NAME**='DSA' or **C_NAME**='DBMS'))); The innermost query will return a set with members C1 and C3. Second inner query will return those **S_ID**s for which **C_ID** is equal to any member of set (C1 and C3 in this case) which are S1, S2 and S4. The outermost query will return those **S_ID**s where **S_ID** is not a member of set (S1, S2 and S4). So it will return S3.

- **Co-related Nested Queries:** In co-related nested queries, the output of inner query depends on the row which is being currently executed in outer query. e.g.; If we want to find out **S_NAME** of **STUDENT**s who are enrolled in **C_ID** 'C1', it can be done with the help of co-related nested query as: Select **S_NAME** from **STUDENT** S where EXISTS (select * from **STUDENT_COURSE** SC where S.**S_ID**=SC.**S_ID** and SC.**C_ID**='C1'); For each row of **STUDENT** S, it will find the rows from **STUDENT_COURSE** where S.**S_ID** = SC. **S_ID** and SC.**C_ID**='C1'. If for a **S_ID** from **STUDENT** S, atleast a row

exists in **STUDENT_COURSE** SC with **C_ID**='C1', then inner query will return true and corresponding **S_ID** will be returned as output.

EXAMPLE IN SQL CODE:

```
SELECT StudentName
FROM Students
WHERE Student_ID IN (
SELECT Student_ID
FROM Grades
WHERE Subject = 'Mathematics' AND Score > 90
);
```